Module 5

Content

- High Performance Computing on Cloud
- Load Balancing and Resource utilization in Cloud
- Mini Project on implementation of Cloud

High Performance Computing

- HPC is the practice of aggregating computing resources to gain performance greater than that of a single workstation, server, or computer.
- HPC can take the form of custom-built supercomputers or groups of individual computers called **clusters**. HPC can be run on-premises, in the cloud, or as a hybrid of both.
- Each computer in a cluster is often called a **node**, with each node responsible for a different task.
- **Controller nodes** run essential services and coordinate work between nodes.
- **Interactive nodes**, or login nodes act as the hosts that users log in to, either by graphical user interface or command line.
- **Compute nodes** execute the computations. Algorithms and software are run in parallel on each node of the cluster to help perform its given task.

High Performance Computing

- HPC typically has three main components: **compute, storage, and networking**.
- HPC allows companies and researchers to aggregate computing resources to solve problems that are either too large for standard computers to handle individually or would take too long to process. For this reason, it is also sometimes referred to as supercomputing.
- HPC is used to solve problems in academic research, science, design, simulation, and business intelligence.
- HPC's ability to quickly process massive amounts of data powers some of the most fundamental aspects of today's society, such as the capability for banks to verify fraud on millions of credit card transactions at once, for automakers to test your car's design for crash safety, or to know what the weather is going to be like tomorrow.

High Performance Computing

- High performance computing has three main components:

Compute

Network

Storage

- In basic terms, **the nodes (compute)** of the HPC system are connected to other nodes to run algorithms and software simultaneously, and are then **connected (network)** with data **servers (storage)** to capture the output.
- As HPC projects tend to be large and complex, the nodes of the system usually have to exchange the results of their computation with each other, which means they need fast disks, high-speed memory, and low-latency, high-bandwidth networking between the nodes and storage systems.

High Performance Computing on Cloud

- HPC can typically be broken down into two general design types: **cluster computing and distributed computing.**
- **Cluster computing:** Parallel computing is done with a collection of computers (clusters) working together, such as a connected group of servers placed closely to one another both physically and in network topology, to minimize the latency between nodes.
- **Distributed computing:** The distributed computing model connects the computing power of multiple computers in a network that is either in a single location (often on-premises) or distributed across several locations, which may include on-premises hardware and cloud resources.

High Performance Computing

- HPC clusters can be distinguished between **homogeneous vs heterogeneous hardware** models.
- **In homogenous clusters**, all machines have similar performance and configuration, and are often treated as the same and interchangeable.
- **In heterogeneous clusters**, there is a collection of hardware with different characteristics (high CPU core-count, GPU-accelerated, and more), and the system is best utilized when nodes are assigned tasks to best leverage their distinct advantages.

High Performance Computing

How do HPC jobs work?

- Workloads in an HPC environment typically come in two different types: **loosely coupled and tightly coupled.**

- **Loosely coupled workloads** (often called parallel or high throughput jobs) consist of independent tasks that can be run at the same time across the system. The tasks may share common storage, but they are not context-dependent and thus do not need to communicate results with each other as they are completed. An example of a loosely coupled workload would be rendering Computer Generated Imagery (CGI) in a feature film, where each frame of the video is rendered independently of the other frames, despite them sharing the same input data like backgrounds and 3D models.

High Performance Computing

How do HPC jobs work?

- Workloads in an HPC environment typically come in two different types: **loosely coupled and tightly coupled.**

- **Tightly coupled workloads** consist of many small processes, each handled by different nodes in a cluster, which are dependent on each other to complete the overall task. Tightly coupled workloads usually require low-latency networking between nodes and fast access to shared memory and storage. Interprocess communication for these workloads is handled by a Message Passing Interface (MPI), using software such as OpenMPI and Intel MPI. An example of a tightly coupled workload would be weather forecasting, which involves physics-based simulation of dynamic and interdependent systems involving temperature, wind, pressure, precipitation, and more. Here, each cluster node may compute partial solutions to different weather factors, contributing to the overall forecast.

High Performance Computing on Cloud

- **HPC can be performed on-premises with dedicated equipment, in the cloud, or a hybrid of each.**
- **HPC in the cloud offers the benefit of flexibility and scalability without having to purchase and maintain expensive dedicated supercomputers.**
- **HPC in the cloud provides all the necessary infrastructure needed to perform large, complex tasks such as data storage, networking solutions, specialized compute resources, security, and artificial intelligence applications.**
- **Workloads can be performed on demand, which means that organizations can save money on equipment and time on computing cycles, only using the resources they need, when they need them.**

High Performance Computing on Cloud

Some common considerations when choosing to run HPC in the cloud include:

- **Latency and bandwidth:** With the amount of data running in HPC workloads, cloud providers need to provide robust networking capabilities (>100 GB/s) with low latency.
- **Performance:** HPC in the cloud works best with providers that constantly update systems to optimize performance, especially in computer processors, storage solutions, and networking capabilities.
- **Sustainability:** HPC is a resource intensive form of computing, requiring much more electricity than normal workloads. On-premises high performance computers can cost millions of dollars a year in energy. Public clouds that prioritize renewable energy—such as Google Cloud—can mitigate the energy impacts of HPC.
- **Storage:** Given the size of most HPC tasks, scalable data storage is an important consideration when running HPC workloads. Cloud providers that can easily store and manage large amounts of data (such as through Google's managed storage services , including Filestore, or Managed Lustre) have an advantage in HPC.

High Performance Computing on Cloud

- **Speed and performance:** High performance computing can process data and tasks much faster than a single server or computer. Tasks that could take weeks or months on a regular computing system can take hours in HPC.
- **Flexibility and efficiency:** With HPC in the cloud, workloads can be scaled up or down depending on need. With a robust internet connection, HPC can be accessed from anywhere on the globe.
- **Cost savings:** Because of the speed, flexibility, and efficiency of HPC in the cloud, organizations can save time and money on computing resources and labor hours.
- **Fault tolerance:** If one node of an HPC cluster fails, the system is resilient enough that the rest of the system does not come crashing down. Given the large and complex tasks performed by HPC, fault tolerance is a big advantage.
- **Accelerated R&D:** HPC provides an advantage to companies performing research and development by speeding up the results of data-intensive projects, such as pharmaceutical modelling, designing new machines and parts, or simulating experiments to reduce physical testing.

High Performance Computing on Cloud

- **Security:** A cloud provider with a privately managed global infrastructure ensures that data and applications are the least exposed to the public internet. Virtual private cloud (VPC) networks enable connectivity between nodes and can configure firewalls for HPC applications. Confidential Computing features allow encryption in use, as well as encryption at rest and in flight.
- **Initial cost:** On-premises HPC clusters and supercomputers have high initial costs. On-premises HPC would be out of reach for most organizations after factoring in the cost of equipment, labor, software, and configuration.
- **Energy consumption:** The energy costs of on-premises supercomputer installations can be large. For environmentally and cost conscious companies, HPC energy consumption can be sustainable by running HPC on the world's cleanest cloud.
- **Maintenance:** HPC runs best when on the latest generation of hardware and optimized software. Keeping an on-premises HPC cluster or supercomputer up to date to ensure optimal performance can quickly become a large and ongoing expense.

High Performance Computing on Cloud

Why is HPC important?

- The modern ecosystem is flooded with data and compute-intensive tools to analyze it. HPC allows businesses and organizations to process all of that data in a timely manner, fueling new insights, innovations, and scientific discoveries. This allows companies to forecast business scenarios, predict market fluctuations, and make recommendations. The medical field, meanwhile, is being transformed with easy access to HPC in the cloud, helping to model potential outbreaks, decode the genome of cancerous cells, and understand how diseases evolve.

- In short, HPC is accelerating the rate at which scientific, technological, and business progress is made, helping humanity craft a more prosperous future.

High Performance Computing on Cloud

Use Cases for HPC

- **Research:** HPC is used in academic and scientific research to perform analysis and computation of large datasets, such as astronomical imaging data, material design, drug discovery, or proteins dynamics.

- **Simulation:** HPC is used to simulate physical scenarios such as automobile collisions, airflow over airplane wings or inside of engines, or how potential new drugs interact with human cells.

- **Design:** Manufacturers often use HPC to design and simulate new products such as planes, phones, and cars in software before building physical prototypes, accelerating the manufacturing process. Computer chip manufacturers use HPC to model new chip designs before prototyping.

High Performance Computing on Cloud

Use Cases for HPC

- **Optimization:** HPC can help optimize large and difficult datasets, such as financial portfolios or the most efficient routes for shipping and logistics.

- Forecasting: HPC can take large, complex datasets and make predictions in a timely manner. Many aerospace companies use HPC to predict when their machines will require maintenance. Weather forecasting is performed with high performance computing, allowing meteorologists to predict storms, and model climate change.

- Data analysis: HPC can analyze the largest of datasets. When applied to data analytics, HPC can help distribute processing to respond to queries faster, enabling real-time credit card fraud detection and other time-critical processes.

High Performance Computing on Cloud

Related products and services

Google Cloud offers a variety of products and services to help organizations make the most of HPC in the cloud.



Compute Engine

Secure and customizable compute service that lets you create and run virtual machines on Google's infrastructure.





Cloud Storage

Object storage for companies of all sizes. Store any amount of data. Retrieve it as often as you'd like.





Google Cloud networking products

Planet-scale networking for a smart and connected world.





Batch

Fully managed batch service to schedule, queue, and execute batch jobs on Google's infrastructure.





Google Kubernetes Engine

A simple way to automatically deploy, scale, and manage Kubernetes.



High Performance Computing on Cloud

Cloud Component	HPC Use	Simple Example
Compute Engine	Provides powerful virtual machines (VMs) for computation	🧬 **Drug discovery:** Run molecular simulations on multiple high-CPU/ GPU VMs to find promising drug molecules.
Cloud Storage	Stores very large HPC datasets and results	🌦️ **Weather forecasting:** Store huge satellite/weather datasets and simulation results for analysis.
Google Cloud Networking	Provides fast, low-latency communication between HPC machines	🚙 **Car crash simulation:** Multiple VMs exchange simulation data quickly while working on different parts of the crash model.
Batch	Automatically schedules and runs large numbers of HPC jobs	🔬 **Research:** Submit 1,000 molecular simulations; Batch distributes them across available VMs and runs them automatically.
Google Kubernetes Engine (GKE)	Manages containerized HPC workloads and scales resources	🤖 **AI research:** Run multiple GPU-based ML experiments in containers and automatically allocate resources to different experiments.

Load Balancing & Resource Utilization in Cloud

Distributing work to keep cloud systems fast, efficient and available

Department of Computer Science and Engineering

Why Load Balancing Matters

1 **Performance**

Spreads requests so no server becomes a bottleneck

2 **Availability**

Reroutes traffic away from failed or unhealthy nodes

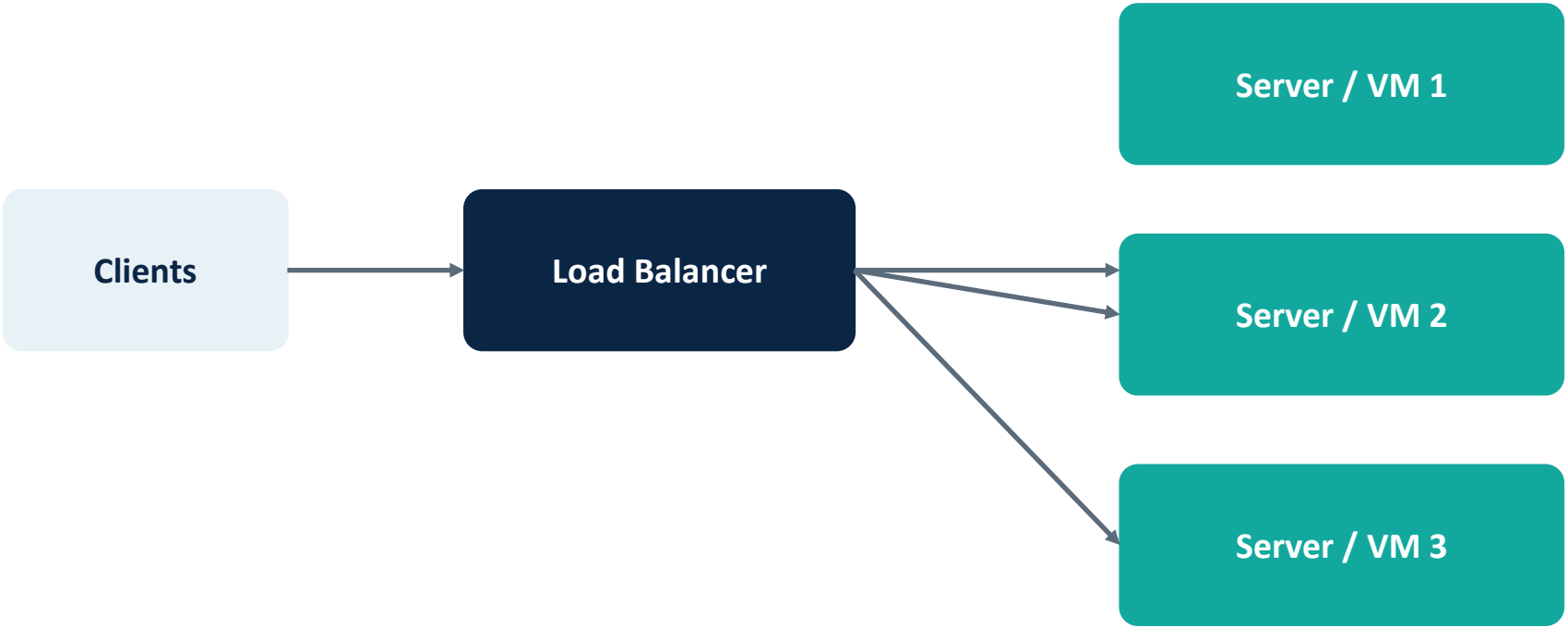
3 **Scalability**

Adds or removes capacity as demand changes

4 **Cost efficiency**

Higher utilization means fewer idle resources to pay for

How It Works



Health checks, session persistence and routing policy decide where each request goes.

Layer 4 vs Layer 7 Load Balancing

Layer 4 (Transport)

Routes on IP address and port

- Very fast, low overhead
- Protocol-agnostic (TCP/UDP)
- No view into request content
- Good for raw throughput

Layer 7 (Application)

Routes on HTTP content

- Path, host and header-based routing
- SSL termination and caching
- Cookie-based session affinity
- Higher CPU cost, richer control

Load Balancing Algorithms

Type	Algorithm	Best suited for
Static	Round Robin	Uniform servers, similar requests
Static	Weighted Round Robin	Servers with unequal capacity
Dynamic	Least Connections	Long-lived, uneven sessions
Dynamic	Least Response Time	Latency-sensitive services
Hash-based	IP Hash	Session affinity without state sharing
Adaptive	Predictive / ML-based	Bursty, forecastable workloads

Algorithm Mechanics

Weighted Round Robin

`share_i = w_i / Σw`

Weights reflect capacity, e.g. 4-core = 2, 2-core = 1.

Least Connections

`argmin(active_i / w_i)`

Tracks open connections; weight-normalized for mixed hardware.

Power of Two Choices

`sample 2, pick lighter`

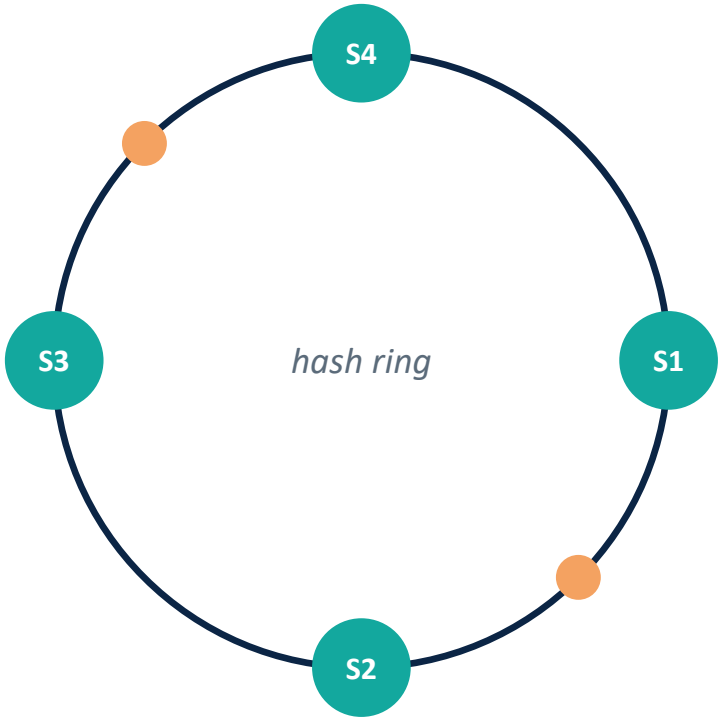
Avoids scanning every server; max load falls sharply vs pure random.

Peak EWMA Latency

`ewma_rtt × (inflight+1)`

Favors fast, lightly loaded backends and adapts to slow nodes.

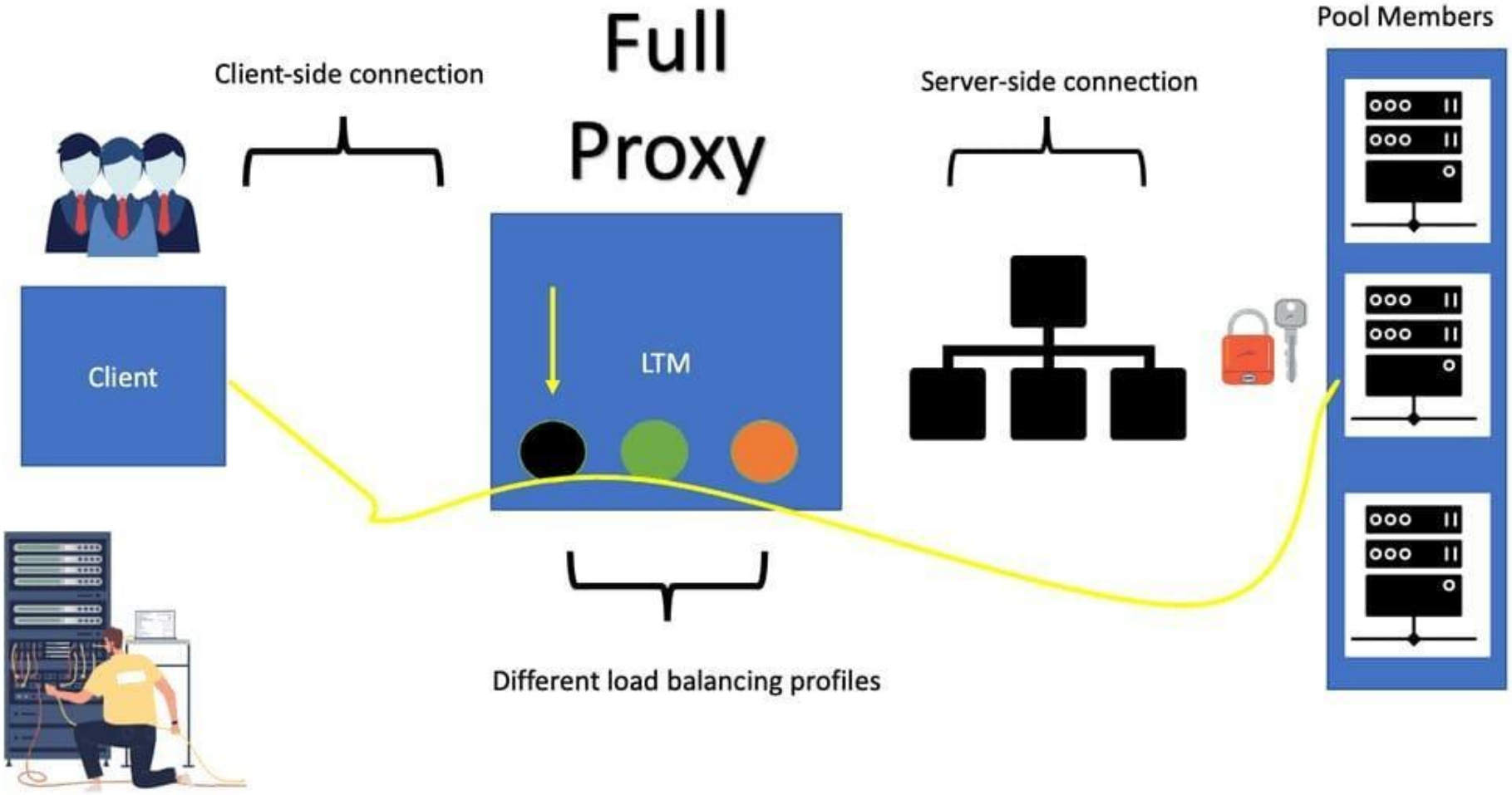
Consistent Hashing



- hash(key) mod N: adding one server remaps most keys
- Ring: each key goes to the next server clockwise (amber dots = keys)
- Adding or removing a node moves only about 1/N of keys
- Virtual nodes even out uneven arcs
- Maglev-style lookup tables trade a little balance for fast, stable mapping

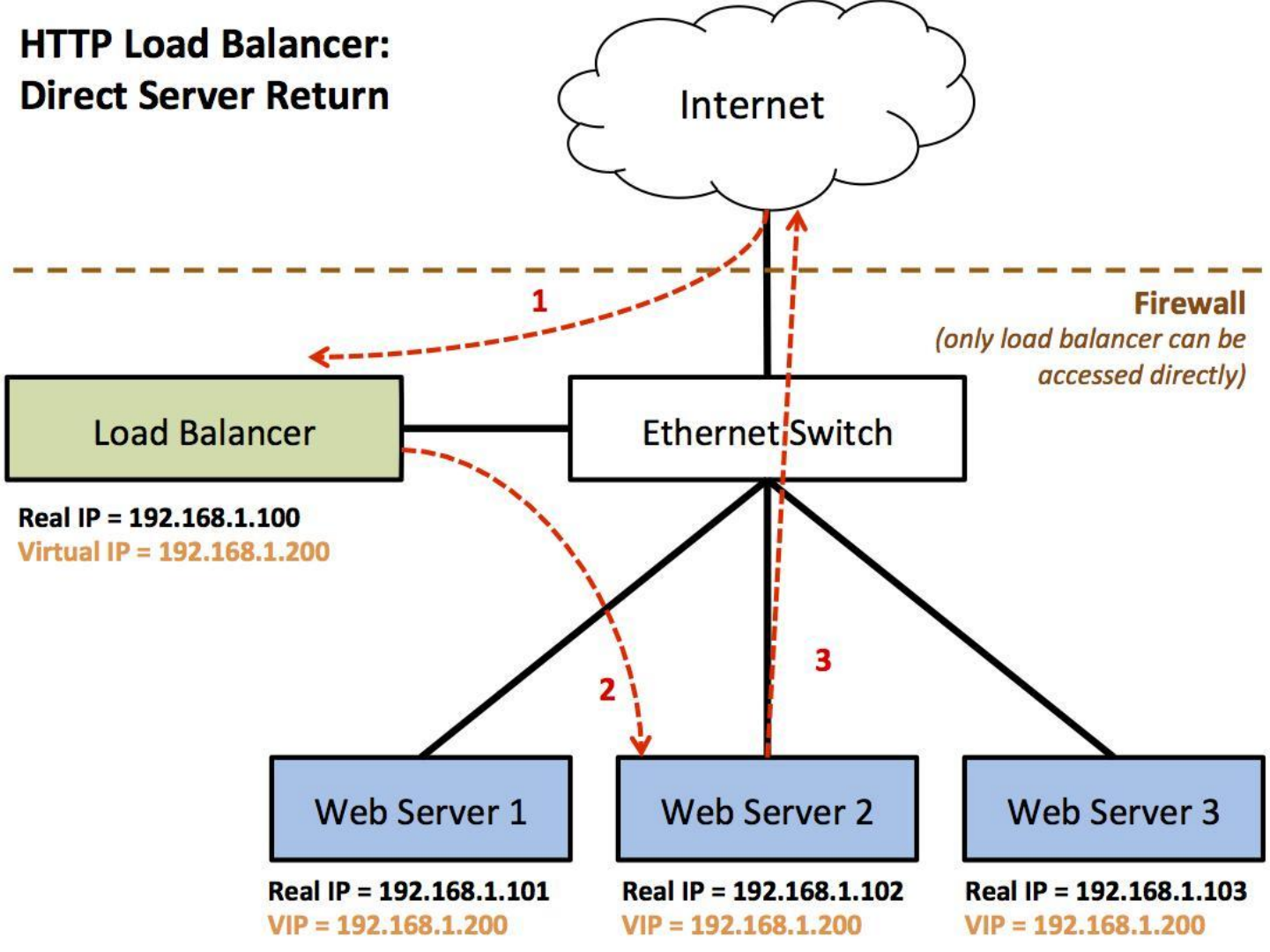
Forwarding Modes and Balancer Topologies

Mode	How it works	Trade-off
Proxy (full NAT)	LB terminates the client connection and opens a new one to the backend	Full L7 control; all traffic in both directions crosses the LB
Direct Server Return	LB rewrites the destination MAC; backend replies straight to the client	High throughput; no L7 features, VIP on loopback
IP tunneling	LB encapsulates packets (IP-in-IP/GRE) to backends	Works across subnets; adds MTU overhead
ECMP + anycast	Routers spread flows over LB instances by 5-tuple hash	Scales LBs horizontally; flows can remap when instances change
DNS / GSLB	Name resolves to different regional addresses	Global steering; failover limited by TTL


Full
Proxy
Client-side connection
Server-side connection
Pool Members
Client
LTM
Different load balancing profiles

NGINX AS REVERSE PROXY & LOAD BALANCER




HTTP Load Balancer:
Direct Server Return
Internet
1
Firewall
(only load balancer can be accessed directly)
Load Balancer
Ethernet Switch
Real IP = 192.168.1.100
Virtual IP = 192.168.1.200
2
3
Web Server 1
Web Server 2
Web Server 3
Real IP = 192.168.1.101
VIP = 192.168.1.200
Real IP = 192.168.1.102
VIP = 192.168.1.200
Real IP = 192.168.1.103
VIP = 192.168.1.200

Health Checks and Failure Detection

Parameter	Example
Probe interval	10 s
Probe timeout	5 s
Unhealthy threshold	3 failures
Healthy threshold	2 successes

- Active probes: TCP connect or HTTP GET /healthz expecting 200
- Passive checks: eject nodes after consecutive 5xx or timeouts (outlier detection)
- Connection draining: let in-flight requests finish before removal
- Keep health endpoints cheap; deep dependency checks can cause cascading ejections

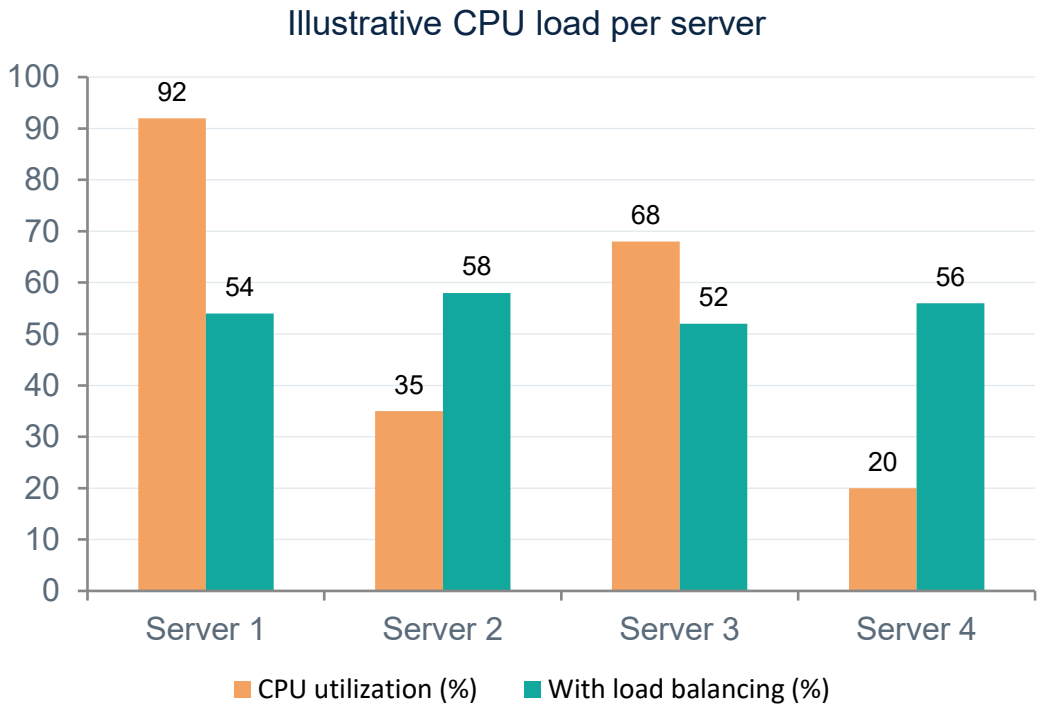
Worst-case detection ≈ interval × unhealthy threshold ≈ 30 s

Session Persistence and State

Method	Mechanism	Drawback
Source-IP affinity	Hash of client IP picks the backend	Clients behind one NAT pile onto one server
LB-inserted cookie	LB sets a cookie naming the backend (L7)	Session lost if that backend is removed
App session ID	LB routes on an application cookie	LB must understand the app protocol
Stateless + shared store	Sessions kept in Redis or a database; any node serves	Extra latency; the store becomes a dependency

Prefer stateless services: balancing, scaling and failover all become simpler.

Impact on Resource Utilization



Without balancing
Hot spots and idle nodes coexist, wasting capacity.

With balancing
Load evens out, so the same fleet handles more traffic.

Illustrative data

Quantifying Load Imbalance

Imbalance factor

`max(load) / mean(load)`

1.0 means perfectly even.

Jain's fairness index

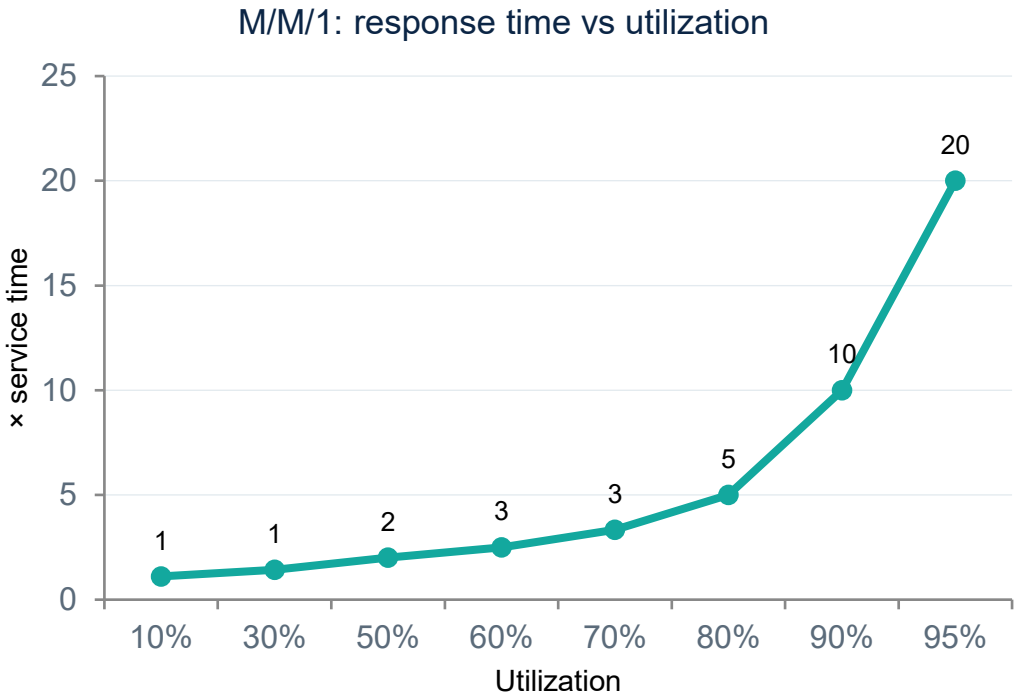
$J = (\Sigma x)^2 / (n \cdot \Sigma x^2)$

Ranges from 1/n (worst) to 1 (even).

Metric	Before	After
Max / mean	1.71	1.05
Jain's index	0.79	1.00
Std dev (pp)	28.1	2.2

Computed from the illustrative CPU values on the utilization chart.

Why Headroom Matters: A Queueing View



$W = (1/\mu) / (1 - \rho)$

Little's Law: $L = \lambda \cdot W$

Delay grows without bound as ρ approaches 1, so plan capacity for peaks, not averages.

Challenges and Optimization Techniques

Challenges

- Unpredictable, bursty traffic
- Heterogeneous VMs and hardware
- Balancer as a single point of failure
- Migration overhead and SLA limits

Techniques

- Auto-scaling with health checks
- VM migration and consolidation
- Multi-tier and global balancing
- Energy-aware and ML-driven scheduling

Load Balancing in Major Clouds

1	**AWS**	**Elastic Load Balancing** Application, Network and Gateway load balancers
2	**Microsoft Azure**	**Load Balancer, Application Gateway** Layer 4 balancing plus Layer 7 gateway; Front Door for global routing
3	**Google Cloud**	**Cloud Load Balancing** Global and regional balancing behind a single anycast IP

Service names as commonly offered; check provider documentation for current options.

Measuring Resource Utilization

CPU

Processing load per VM or container

Memory

Working set vs allocated RAM

Network I/O

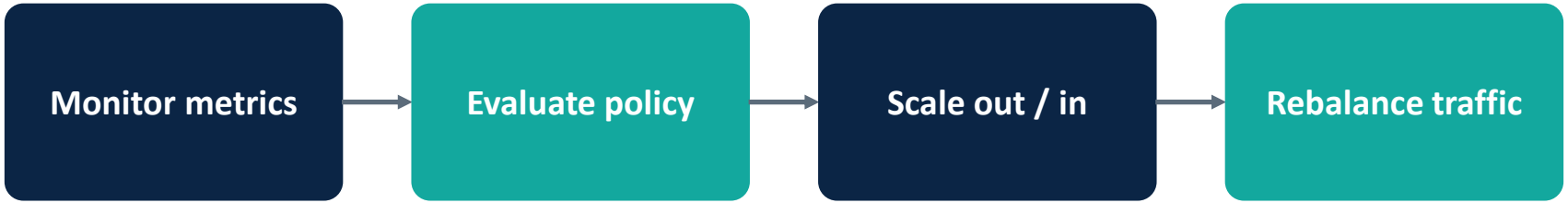
Bandwidth and packets per second

Response time

Latency and request queue depth

Goal: keep average utilization high enough to avoid waste, with headroom to absorb spikes and meet SLAs.

Auto-Scaling: The Feedback Loop



Scale out when load crosses an upper threshold; **scale in** when it stays low, using cooldown periods to avoid flapping.

Auto-Scaling: Target Tracking Math

$$\text{desired} = \text{ceil}(\text{current} \times \text{metric} / \text{target})$$

4 instances at 80% CPU, target 50% → ceil(4 × 1.6) = 7

Cooldown

Wait after a scaling action before acting again.

Stabilization

Use recent recommendations to damp flapping, especially on scale-in.

Warm-up

Exclude new instances from metrics until they are ready.

VM Placement and Live Migration

Placement as bin packing

- NP-hard: pack VMs onto the fewest hosts under CPU, RAM and network limits
- Heuristics: First Fit, Best Fit, First Fit Decreasing
- Metaheuristics: genetic algorithm, PSO, ant colony
- Trade-off: consolidation saves energy but raises contention risk

Live migration (pre-copy)

- Copy memory pages while the VM keeps running
- Re-copy pages dirtied meanwhile, in rounds
- Pause briefly; send last dirty pages and CPU state
- Resume on the destination host
- Cost: network and CPU; high dirty-page rates prolong migration

Best Practices

1. Run balancers redundantly across zones
2. Configure health checks and graceful draining
3. Design services to be stateless where possible
4. Terminate SSL at the balancer and monitor continuously
5. Load test and tune scaling thresholds regularly

Key Takeaways

- Load balancing keeps cloud services responsive and available
- The right algorithm depends on workload and server heterogeneity
- Better utilization cuts cost and energy use
- Future direction: adaptive, ML-based and energy-aware balancing

Thank you | Questions?